Data Warehousing Project
Software Project Management Plan

Data Warehousing Team
Alex Anthony
John Cortez
Daemon Lewis
Hayden Nikkel
Jackson Rosco

Data Warehousing Project

January 22nd, 2026
1.0

_____
[Project Lead Signature Block]

_____

[Project Manager Signature Block]

1 1

1.1 1
1.2 1
1.3 1
1.4 1
1.4.1 1
1.4.2 1

2 4

3 6

3.1 6
3.2 6
3.3 6
3.4 6
3.5 6

4 6

4.1 7
4.2 7
4.3 7
4.4 7
4.5 7

5 7

5.1 8
5.2 8
5.3 9

6 10

6.1 10
6.2 11
6.3 11
6.4 11
6.5 11

7 **Error! Bookmark not defined.**

Revision Sheet

Revision Number	Date	Brief summary of changes
001	1/28	Initial baseline draft
002	2/11	Polished sections of Introduction
003	3/7	Revised Project Organization

1 Introduction

1.1 Project Overview

This subproject, focused on implementing the “ELT (and Data Warehouse Implementation)”, is a part of a series of subprojects making up the "Algorithmic Trading and Short-term Forecast for Financial Time Series with Machine Learning Models” project. In essence, this project will transform automated data collection databases into a formatted data warehouse for use in the “BI/XGBoost training and testing implementation”. It should do so in a manner that is as efficient as possible for a machine learning algorithm.

The data collected and warehoused by this project is only acquired from the automated data collection of the former subproject, and is only used in the training and implementation of the latter subproject.

This project is developed over seven sprints, and is developed openly as part of a student project.

1.2 Project Deliverables

The project deliverables consist of the completed data warehouse, with proper input and output APIs/systems. These deliverables will be completed and delivered by the end of the seventh sprint.

1.3 Document Overview

The purpose of this document is to act as a management plan for the project and its team. The document will be updated during each sprint as necessary for clarity and use. The document will be edited and revised using document sharing tools, such as Google Drive and Github.

1.4 Acronyms and Definitions

1.4.1 Acronyms

ELT	Extract, Load, Transform
BI	Business Intelligence
XGBoost	eXtreme Gradient Boosting
XP	eXtreme Programming
CI/CD	Continuous Integration/Continuous Delivery

1.4.2 Definitions

data warehouse	"A data warehouse is a type of data management system that is designed to enable and support business intelligence (BI) activities, especially analytics. Data warehouses are solely intended to perform queries and analysis and often contain large amounts of historical data. The data within a data warehouse is usually derived from a wide range of sources such as application log files and transaction applications." [Oracle]
configuration management	"A discipline applying technical and administrative direction and surveillance to: identify and document the functional and physical characteristics of a configuration item, control changes to those characteristics, record and report change processing implementation status, and verify compliance with specified requirements." [IEEE90]
customer	"The individual or organization that specifies and accepts the project deliverables. The customer may be internal or external to the parent organization of the project, and may or may not be the end user of the software product. A financial transaction between the customer and developer is not necessarily implied." [IEEE87]
function	"An activity that spans the entire duration of a software project. Examples of project functions include project management, configuration management, quality assurance, and verification and validation." [IEEE87]
function points	—Metric used to determine size of effort for a project based on the "functionality" or "utility" of the project.
Function Point Method	—Method proposed by [Albrecht79] to determine the size of effort for completing a project based on an empirical relationship between the software's information domain and assessments of the software's complexity.
plan	"A detailed scheme, program, or method worked out beforehand for the accomplishment of an objective." [Heritage85] —Defined set of procedures and the required resources to implement a policy.
policy	"A course of action, guiding principle, or procedure considered to be expedient." [Heritage85] —Corporate strategy, defines high-level goals.
process	"A sequence of steps performed for a given purpose." [IEEE90] —Activities and interfaces used to implement plan.
project	—unit of work to meet a specific customer requirement. Includes all tasks, activities, and functions necessary to meet the requirements.

project agreement	"A document or set of documents agreed to by the designated authority for the project and the customer. Documents in a project agreement may include some or all of the following: a contract, a statement of work, system engineering specifications, user requirements specification, functional specifications, the software development plan, a business plan, or a project charter." [IEEE87]
project deliverables	"The work product(s) to be delivered to the customer. The quantities, delivery dates, and delivery locations are specified in the project agreement." [IEEE87]
quality assurance	"(1) A planned and systematic pattern of all actions necessary to provide adequate confidence that an item or product conforms to established technical requirements." [IEEE90] "(2) A set of activities designed to evaluate the process by which products are developed or manufactured. " [IEEE90]
review	—A process or meeting during which a work product, or set of work products, is presented to program personnel, managers, users, customers, or other interested parties for comment or approval. Types include code review, design review, formal qualification review, requirements review, test readiness review.
software	"Computer programs, procedures, and associated documentation and data pertaining to the operation of a computer system." [IEEE90]
software engineering	"The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software." [IEEE90]
software project management	"The process of planning, organizing, staffing, monitoring, controlling, and leading a software project." [IEEE87]
software quality assurance	—See quality assurance.
specification	"A document that specifies, in a complete, precise, verifiable manner, the requirements, design, behavior, or other characteristics of a system or component, and, often, the procedures for determining whether these provisions have been satisfied." [IEEE90]
statement of work	—Description of all the work required to complete a project, which is provided by the customer.
system requirements	—A condition or capability that must be met or possessed by a system or subsystem component to satisfy a condition or capability needed by a user to solve a problem.

task	"The smallest unit of work subject to management accountability. A task is a well-defined work assignment for one or more project members. The specification of work to be accomplished in completing a task is documented in a work package. Related tasks are usually grouped to form activities." [IEEE87]
techniques	"Technical and managerial procedures that aid in the evaluation and improvement of the software development process." [IEEE90]
work package	"A specification for the work to be accomplished in completing an activity or task. A work package defines the work product(s), the staffing requirements, the expected duration, the resources to be used, the acceptance criteria for the work products, the name of the responsible individual, and any special considerations for the work." [IEEE87]
work product	—Any tangible item that results from a project function, activity, or task. Examples of work products include customer requirements, project plan, design documents, source and object code, user's manuals.
sprint	"They are fixed length periods of work that last one month or less to create consistency and ensure short iterations for feedback in order to inspect and adapt both how work is done and what is being worked on." [Scrum.org]

2 References

[1] K. Cormier et al., "Data Extraction, Transformation, and Loading (ETL) Process Automation and Data Warehouse Implementation for Algorithmic Trading Machine Learning Modelling*," 2025 IEEE International systems Conference (SysCon), Montreal, QC, Canada, 2025, pp. 1-8, doi: 10.1109/SysCon64521.2025.11014812. keywords: {Automation;Prototypes;Transforms;Predictive models;Data warehouses;Data models;Real-time systems;Data mining;Stock markets;Load modeling}

[2] Software Engineering Project, "Configuration Management," presented to Class, Okanagan College, Kelowna, BC, Canada. January, 2026. [Powerpoint slides]. Available: https://okanagan365-my.sharepoint.com/:p:/g/personal/ykhmelevsky_okanagan_bc_ca/EStXz428bhpOnjVEMSLO_lYBiOBZ4WC8jwnmq4hrlAClKQ

[3] Software Engineering Project, "Configuration Management," presented to Class, Okanagan College, Kelowna, BC, Canada. January, 2026. [Powerpoint slides]. Available: https://okanagan365-my.sharepoint.com/:p:/g/personal/ykhmelevsky_okanagan_bc_ca/EStXz428bhpOnjVEMSLO_lYBiOBZ4WC8jwnmq4hrlAClKQ

[4] J. Drenka et al., "Algorithmic Trading Subsystems Data Collection and Transformation Process Automation Using DRI for Machine Learning Modelling and Forecasting," Research paper, Okanagan College / UBCO / UFV, Kelowna, Canada, 2026.

[5] K. Cormier et al., "Algorithmic Trade Forecasting DBMS Implementation for AI and ML within Digital Research Alliance of Canada Advanced Research Computing (ARC) Platforms," Research paper, Okanagan College / UBCO / UFV, Kelowna, Canada, 2026.

[6] IEEE Std 1058-1998, "IEEE Standard for Software Project Management Plans," Institute of Electrical and Electronics Engineers, New York, NY, 1998.

[7] IEEE Std 829-1998, "IEEE Standard for Software Test Documentation," Institute of Electrical and Electronics Engineers, New York, NY, 1998.

[8] G. van Rossum et al., "PEP 8 – Style Guide for Python Code," Python Enhancement Proposals, 2001. Available: https://peps.python.org/pep-0008/.

[9] D. Goodger and G. van Rossum, "PEP 257 – Docstring Conventions," Python Enhancement Proposals, 2001. Available: https://peps.python.org/pep-0257/.

3 Project Organization

3.1 Organizational Policies

Organizational policies coincide with the Okanagan College student conduct guidelines, as well as the guidelines present in the course syllabus.

3.2 Process Model

This project uses the Agile process model, with each student in the 6-person team having equal responsibility for the project management.

3.3 Organizational Structure

There is one Scrum Master, and one Product Owner. Each student belongs to an XP Coding Pair. The professor overseeing the project is the only stakeholder.

3.4 Organizational Boundaries and Interfaces

As previously stated, this project is a middle party between the automated data collection project software and the BI/XGBoost training and testing implementation project software. As such, the project teams of all three subprojects are working closely together in tandem to create the final project. The project owner is updated every sprint via submitted documentation on the project process.

Each team member is responsible for the configuration management, quality assurance, and all other administrative and managerial documentation.

3.5 Project Responsibilities

The project team is required to successfully complete each sprint, as well as all tasks within each sprint, and all documentation to successfully complete the project.

4 Managerial Process

Management objectives of the project is to keep all sprints on track for completion, as well as prioritize completion of documentation. Risk management will be done through a separate document. The project will be monitored through a distributed source control system, and the staffing plan is static, with the 6 team members of the project.

4.1 Management Objectives and Priorities

The goals of the management of the project will align with the Agile methodology, with the Agile process being used as a guide throughout the project.

4.2 Assumptions, Dependencies, and Constraints

This project assumes that each subproject will be completed and worked on in parallel. This project is constrained by each subproject's features and completion.

4.3 Risk Management

Completed risk management forms will be attached to this section in a subsequent revision.

4.4 Monitoring and Controlling Mechanisms

Each sprint will have reports on work completed, as well as sprint retrospectives and other reviewing. After these reviews, subsequent sprints will consider these reviews when planning work and goals.

4.5 Staffing Plan

The staffing plan consists of the 6 students who volunteered to work as a team on this subproject.

5 Technical Process

The technical process of developing the project is a sequence of design, test creation, and development to pass said tests. Tools to be used with testing are unit test libraries and software to create and analyze the software easily. Automated testing via CI/CD will also be used.

Software documentation will be prioritized in this project, with comprehensive documentation maintained every sprint. Quality assurance will be done with the client directly through acceptance tests, as well as through integration, regression and unit tests of the system.

5.1 Tools, Techniques, and Methods

Desktop environments constitute the computing systems used to develop and design the data warehouse system, although enterprise servers will be used to deploy the project. Programming languages include, but are not limited to: Python, PL/SQL, etc. Github will be used for source control, with an object-oriented design focus.

Team structures consist of XP Pairs, switching teammates every sprint. Due to the small team size, the team structures remain mostly static.

5.2 Software Documentation

The development team will be responsible to produce the following documents.

Project Management Documents

- Software Project Management Plan
  - Updated per sprint
  - Sprint reports and retrospectives
- Requirements and Design Documents
  - Data warehouse requirements: API endpoints for data collection and ML access, support for 5-15 minute Financial Modeling Prep API data
  - Data Model Documentation: Entity-relationship diagrams, star schema descriptions (fact tables, dimension tables), column definitions, transformation descriptions
  - ELT Pipeline Design: Flow diagrams, SQL transformation logic
- Code Documentation
  - Python scripts
  - SQL files
- API Documentation
  - Warehouse API specification: Endpoints, request/response formats, authentication, ML integration
- Test Documentation aligned with IEEE Std 829-1998
  - Test Plan: Scope, approach, data validation schedule .
  - Test Cases/Specifications: Data integrity, completeness, performance.
  - Test Procedures: Scripts for running test validation
  - Test Summary Reports: Pass/Fail metrics on a per-sprint basis
- Standard Operation Documentation
  - User Guide: How to query the warehouse
  - Deployment Guide: Setup, monitoring, maintenance

Milestones, Baselines, Reviews, Sign-offs

- Milestones
  - See Section 6.1 in this document
- Baselines
  - Versioning done on GitHub with incremental versions per-sprint
- Reviews
  - Extreme Programming pairs for peer/code review
  - Sprint retrospectives for documentation feedback
  - Approval by professor (client) at the end of each sprint.
- Sign-offs
  - Scrum master to sign-off on drafts and major baselines

Style Guide, Naming Conventions, Documentation Formats

- Python
  - Follow PEP 8 guidelines
- Format/Tools
  - Markdown on Github
  - Diagrams: Draw.io
  - API: FastAPI specification
  - Version Control: Git with branches

Schedule and Resource Summary

- Effort Allocation
  - 4-12 hours per team member weekly
- Schedule
  - Incremental updates per sprint
- Resources
  - Google Drive for documentation collaboration, GitHub for version storage

5.3 Project Support Functions

The project will be supported by a suite of documentation, including a business case document, software specification document, use case specifications, configuration management plan, change request documentation as well as user support documents and developer's guide.

6 Work Packages

Work packages consist of biweekly sprints, with each sprint iterating on the previous sprint's completed/delivered work. Resources, including budget, are provided by team members or the college. The project schedule will follow the sprints up to Sprint 7.

6.1 Work Packages

[Specify the work packages for the activities and tasks that must be completed in order to satisfy the project agreement. Each work package should be uniquely identified; identification may be based on a numbering scheme and descriptive titles. A diagram depicting the breakdown of activities into subactivities and tasks may be sued to depict hierarchical relationships among work packages.]

Each sprint and its requirements for completion are available on the course website. As each sprint is added to the website, it will be shown here.

Sprint 1: Inception

- Sprint Deliverables
    - Scrum Stand Ups
    - Sprint Review Meeting
    - Backlog Grooming Meeting
    - Sprint Planning Meeting
    - User Stories/Tasks Meeting

- Project Technical Documentation and Project Analysis
    - Initial Business Case
    - Initial Software Specification
    - Initial Use Case Specifications
    - Initial Configuration Management Plan
    - Initial Change Request Document
    - Initial User Support Documentation
    - Initial Developer's Guide

Sprints 2-7 (Changes will be noted):

- Sprint Deliverables
    - Scrum Stand Ups
    - Sprint Review Meeting
    - Backlog Grooming Meeting
    - Sprint Planning Meeting
    - User Stories/Tasks Meeting

- Project Technical Documentation and Project Analysis
    - Business Case
    - IEEE Project Specification
    - Use Case Documentation
    - Configuration Management Plan
    - Change Request Documentation

    - User Support Documentation

- Software Design And Development
    - Design Document
    - Source Code
    - Testing Code

- Test Suite
    - Acceptance Test Results
    - Unit Tests
    - Integration Tests
    - Reports from CI system

6.2 Dependencies

For data to enter the data warehouse, the data warehouse must be fully designed and implemented. The data warehouse must be able to load and transform data from another database, which needs to be designed first for the data warehouse to exist.

External events will have a major effect on the project, as the development of each subproject depends on each other. This subproject will have to use mock data or other techniques to test and develop our subproject in the event that the other subprojects are unable to deliver functionality to our project.

6.3 Resource Requirements

Personnel required includes all team members present upon the project inception. Computer time has been estimated to be up to 40 hours per week per person, although this is likely higher than the actual time. Team members will provide their own computer hardware, facilities, travel and maintenance for the project.

6.4 Budget and Resource Allocation

The budget for the project is based on server resources when needed. Other than unexpected costs, the budget is not applicable.

6.5 Schedule

Seven two-week sprints comprise the schedule for the project's development. Major milestones include creation of a warehouse-compatible database reflecting the data collection team's data, a data warehouse design effective for the implementation of XGBoost, and consistent, secure transfer of the data between each subproject.